List of Actions:

🧩 Core

`actions/checkout`

First question you should ask yourself: how will the github action even see your code?

That's where checkout comes in.

You use this at the start of almost every workflow because runners are empty machines. No repo, no files, nothing. The moment you add checkout, your project files exist inside the workflow environment.

Where will you use this?
Any time you want to build, test, scan, or deploy your repository. So basically everywhere.

Think of it like opening your project folder before you start working.

⚙️ Language Setup

Now imagine the runner has your code, but it doesn't know Node, Python, Java, or Go. You have to prepare the environment first.

`actions/setup-node`

You use this when your project runs on JavaScript or TypeScript.

Where will you use it?
Frontend builds, backend APIs, Next.js deployments, anything npm related.

Important mindset:
Every run starts from zero. If you don't install Node, your pipeline breaks.

---

actions/setup-python

You bring this in when the project needs Python.

Where does this show up?
FastAPI services, automation scripts, ML workflows, testing tools.

Your job here is simply telling GitHub which Python version to use so everything stays consistent.

---

actions/setup-java

Whenever you see Spring Boot, Android builds, or Java services, this is your entry point.

Where will you use it?
Backend enterprise apps, CI for Android, Gradle or Maven pipelines.

---

actions/setup-go

If your service is written in Go, this installs the Go toolchain.

Typical use?
Microservices, CLI tools, lightweight APIs.

🧠 Conditional Execution

`dorny/paths-filter`

Let’s say you have a Frontend, backend, infrastructure, everything together.

Do you really want to run all tests every time someone edits a README? No.

This action checks which files changed and decides what should run.

Where will you use it?
Large repos where different teams own different folders.

What this really means is smarter pipelines and faster builds.

🚀 Performance

`actions/cache`

Here’s something students often miss. Runners are temporary. They forget everything after the job ends.

So every install step becomes slow unless you cache dependencies.

Where will you use cache?
Node modules, Python packages, Maven builds, Go modules.

If your pipeline feels slow, caching is usually the first fix.

📦 Artifacts

`actions/upload-artifact`

Sometimes one job produces files that another job needs.

Maybe your build step generates a compiled app. Maybe your tests generate reports.

You upload artifacts so other jobs can access them.

Where will you use this?
Multi-stage pipelines where build and deploy are separate.

`actions/download-artifact`

This is simply the other side of the same flow.

Build job uploads.
Deploy job downloads.

Think of artifacts as temporary storage between jobs.

🐳 Docker / Containers

Now we move into real-world production pipelines.

`docker/setup-buildx-action`

Modern Docker builds use Buildx for better performance and multi-platform support.

Where will you use it?
Any serious Docker build, especially when pushing production images.

---

docker/login-action

Before pushing images, you must authenticate with a registry.

Docker Hub, GitHub Container Registry, AWS ECR, whatever you use.

No login means no push.

---

docker/build-push-action

This is where your app turns into a deployable container.

Instead of running docker build and docker push manually, this action handles it inside CI.

Where will you use it?
Almost every cloud-native deployment pipeline.

☁️ AWS

aws-actions/configure-aws-credentials

Your runner needs permission to talk to AWS.

This action gives it temporary credentials using secure authentication.

Where will you use it?
Deploying to ECS, EKS, Lambda, uploading to S3, running infrastructure automation.

Key lesson here:
Never hardcode AWS keys. Always use secure identity.

🔐 Security

`github/codeql-action`

Security isn't optional anymore.

CodeQL scans your code for vulnerabilities automatically.

Where will you use it?
Pull requests, nightly security scans, compliance-heavy projects.

`aquasecurity/trivy-action`

If you build Docker images, you must scan them.

Trivy checks your images for known vulnerabilities.

Where will you use this?
Right after building containers, before deploying them.

Students should remember:
Shipping unscanned containers is risky.

🚢 Deployment

`appleboy/ssh-action`

This connects to a remote server and runs commands.

Where will you use it?
Connecting to an EC2 instance, restarting Docker containers, running remote scripts.

But understand something important:
It's simple, not always scalable. Great for smaller setups.

```yaml
- name: Deploy to Server
  uses: appleboy/ssh-action@v1.0.3
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ubuntu
    key: ${{ secrets.SERVER_SSH_KEY }}
    script: |
      docker pull myrepo/myapp:latest
      docker stop myapp || true
      docker rm myapp || true
      docker run -d --name myapp -p 80:3000 myrepo/myapp:latest
```

🧠 The Big Picture

Now stop thinking about these as random tools. Think in layers.

First, you get the code.
Then you prepare the environment.
Then you optimize speed.
Then you build.
Then you secure.
Then you automate.
Then you deploy.